

RS-Rently

Distribuirani rent-a-car sustav
zasnovan na mikroservisnoj arhitekturi

Kolegij: Raspodijeljeni sustavi 2025./2026.

Petar Prenc

25. lipnja 2026.

Sadržaj

1	Uvod	3
2	Arhitektura sustava	3
2.1	Auth-service	3
2.2	Booking-service	3
2.3	Damage-service	4
2.4	Mail-worker	4
2.5	GPS-tracker	4
2.6	Frontend	4
2.7	Infrastruktura	5
2.8	Cross-Origin Resource Sharing	5
3	Tehnologije	5
4	Funkcionalnosti	6
4.1	Autentifikacija i autorizacija	6
4.2	Rezervacija vozila	6
4.3	Evidencija šteta	6
4.4	GPS praćenje	6
4.5	Praćenje grešaka	7
5	Komunikacijski protokoli	7
5.1	REST API (HTTP/JSON)	7
5.2	gRPC (Protocol Buffers)	8
5.3	Redis Message Queue	8
6	API Gateway i usmjeravanje prometa	9
7	Horizontalno skaliranje	9
8	Otpornost na pogreške (resilience)	10
8.1	Razina gatewaya	10
8.2	Razina servisa	10
9	Struktura projekta	11
10	Docker i pokretanje	12
11	Dijagram raspodjele – razvojno okruženje	13
12	Dijagram raspodjele – produkcijsko okruženje	13
13	Sekvencijski dijagram	14
14	Konceptualni model domene	15
14.1	Opseg MVP-a	17
14.2	Predviđena proširenja	17
15	Zaključak	18

1 Uvod

RS-Rently je projekt za kolegij Raspodijeljeni sustavi – distribuirani sustav za iznajmljivanje vozila koji sam implementirao kao mikroservisnu arhitekturu. Sustav se sastoji od šest servisa koji pokrivaju različite domene: autentifikaciju, rezervacije, evidenciju šteta, slanje obavijesti, GPS praćenje i korisničko sučelje.

Ideja je bila koristiti više komunikacijskih protokola jer svaki ima drugačiji use case. REST je odabran za standardnu komunikaciju s frontendom, gRPC za prijenos GPS koordinata zbog nižeg overheada, a Redis queue za asinkronu obradu e-mailova – korisniku se odmah vraća potvrda, a slanje maila odvija se u pozadini.

Cijeli sustav pokreće se jednom `docker-compose` naredbom. AWS infrastruktura simulira se lokalno putem LocalStacka, što eliminira potrebu za cloud računom u razvoju.

2 Arhitektura sustava

Svaka poslovna funkcija izdvojena je u zaseban servis. Servisi ne dijele bazu podataka ni memoriju – jedina sprega je mrežna komunikacija. Svih šest servisa i infrastrukturne komponente (Redis, LocalStack te Traefik kao gateway) smješteni su na zajedničku Docker mrežu `rent_network`, unutar koje se pronalaze putem Docker DNS rezolucije. Vanjski promet ne ulazi izravno u servise, nego isključivo kroz gateway (vidi poglavlje o API Gatewayu).

2.1 Auth-service

Auth-service predstavlja ulaznu točku sustava za svakog korisnika. Implementiran je u FastAPI-ju i izlaže dva endpointa: `POST /login` za prijavu te `GET /verify` za provjeru valjanosti tokena. Prijava se temelji na hardkodiranim kredencijalima (`admin/admin`) što je dovoljno za demonstraciju, a u produkcijskoj varijanti zamijenilo bi se bazom korisnika. Nakon uspješne provjere, servis generira JWT token potpisan HS256 algoritmom, s rokom trajanja od jednog sata. Taj token postaje jedini dokaz identiteta – svi ostali servisi očekuju ga u `Authorization` zaglavlju zahtjeva.

2.2 Booking-service

Booking-service prima zahtjeve za rezervaciju vozila na `POST /bookings` endpointu. Prije obrade provjerava *prisutnost* JWT tokena u `Authorization` zaglavlju te valjanost ulaznih parametara (`car_id` i `user_email`). Napomena: u ovoj MVP verziji token se ne validira kriptografski unutar Booking-servisa – provjera potpisa prepuštena je Auth-servisu putem `GET /verify` endpointa kojeg bi gateway ili middleware pozivao u produkcijskom scenariju. Ovakvo pojednostavljivanje svjesno je prihvaćeno radi jasnoće protoka poruka. Za svaku novu rezervaciju generira UUID identifikator, konstruira JSON objekt s podacima o rezervaciji i gura ga u Redis listu `email_queue` naredbom `RPush`. Korisniku odmah vraća potvrdu s identifikatorom rezervacije, dok se daljnja obrada (slanje e-maila) odvija potpuno asinkrono, bez blokiranja odgovora. Ova arhitektonska odluka ključna je za responsivnost sustava – korisnik ne čeka da se e-mail pošalje da bi dobio potvrdu.

2.3 Damage-service

Damage-service zadužen je za evidenciju oštećenja vozila putem uploada fotografija. Izlaže `POST /upload-damage` endpoint koji prima datoteku u `multipart/form-data` formatu. Primljenu datoteku pohranjuje u S3 bucket nazvan `stete-bucket`, pri čemu se koristi LocalStack kao lokalna simulacija AWS-a. Servis također sadrži zasebnu Lambda funkciju (`lambda-function.py`) koja bi na stvarnom AWS-u bila pokrenuta S3 eventom nakon svakog novog uploada – u toj funkciji moguće je implementirati automatsku klasifikaciju štete, obavijest osiguranja ili bilo kakvu drugu obradu.

2.4 Mail-worker

Mail-worker je jedini servis koji ne izlaže nikakav API. Umjesto toga, radi kao pozadinski proces u beskonačnoj petlji koji obrađuje Redis red `email_queue`. Poruke se ne čitaju običnim BLPOP-om, nego se naredbom BRPOPLPUSH atomarno premještaju u zasebnu `processing` listu (*reliable queue*) – time poruka koju obrađuje srušeni worker nije izgubljena, nego se pri sljedećem pokretanju vraća u red. Kada poruka stigne, worker parsira JSON, izvlači e-mail adresu i identifikator vozila te simulira slanje e-maila (`time.sleep(2)` s ispisom u log). Između svake iteracije provjerava i Redis ključ `crash_mail_worker` – mehanizam koji omogućuje da Booking-service na zahtjev izazove simulirani pad workera, što je korisno za demonstraciju Sentry integracije.

Worker je projektiran za pouzdanu obradu: neispravne poruke i one s iscrpljenim brojem pokušaja odlažu se u *dead-letter* red, a skup obrađenih identifikatora osigurava idempotentnost. Ti su mehanizmi detaljno opisani u poglavlju o otpornosti na pogreške. Budući da je servis *stateless* i da se poruke iz reda distribuiraju po načelu *competing consumers*, može se pokrenuti u više replika koje dijele isti red.

2.5 GPS-tracker

GPS-tracker jedini je servis koji ne koristi REST, već gRPC protokol. Sučelje je definirano Protocol Buffers shemom s jednom RPC metodom `SendLocation` koja prima strukturiranu poruku `LocationUpdate` (identifikator vozila, geografska širina i dužina) i vraća praznu poruku `Empty`. Servis koristi `ThreadPoolExecutor` s 10 radnih niti, čime može paralelno obrađivati lokacijske podatke od više vozila.

GPS koordinate su male, strukturirane poruke koje se šalju u visokoj frekvenciji – tu gRPC ima smisla. Binarna serijalizacija i HTTP/2 daju manji overhead od JSON-a, a strogo tipizirana shema znači da se greška poput pogrešno imenovanog polja uhvati ranije, ne tek u produkciji.

2.6 Frontend

Frontend je izgrađen u Vue.js 3 koristeći Composition API. Aplikacija se sastoji od pet komponenti: `Login` za prijavu, `Dashboard` za pregled sustava, `Bookings` za kreiranje rezervacija, `DamageUpload` za upload slika šteta i `CrashControls` za testiranje padova servisa. Vue Router štiti zaštićene rute provjerom prisutnosti tokena u `localStorage` – ako token ne postoji, korisnik se preusmjerava na stranicu za prijavu.

Komunikacija s backendom apstrahirana je kroz centralni modul `api.js` koji kreira tri Axios instance – po jednu za svaki REST servis. Sve tri koriste *relativne* putanje

kroz gateway (`/api/auth`, `/api/booking`, `/api/damage`) umjesto apsolutnih URL-ova s portovima, pa je frontend neovisan o broju i smještaju replika. Interceptor automatski dodaje JWT token u zaglavlje svakog zahtjeva prema Booking i Damage servisima, čime se eliminira potreba za ručnim upravljanjem autentifikacijom u svakoj komponenti.

U produkciji se Vue aplikacija kompajlira u statičke datoteke i servira putem Nginx-a, koji ujedno brine o SPA routingu (sve rute preusmjerava na `index.html`), GZIP kompresiji i cachiranju statičkih resursa.

2.7 Infrastruktura

Dva infrastrukturna servisa čine temelj na kojem ostali servisi grade svoju funkcionalnost. **Redis** (Alpine varijanta) služi kao message broker za asinkronu komunikaciju između Booking-servisa i Mail-workera. Booking-service piše poruke u Redis listu, a Mail-worker ih čita – time se ostvaruje potpuno odvajanje producenta i konzumenta. **LocalStack** simulira AWS ekosustav; u `docker-compose.yaml` aktivirani su `dynamodb` i `s3` servisi, pri čemu trenutna implementacija koristi isključivo S3 (bucket `stete-bucket`) za pohranu fotografija šteta, dok je DynamoDB pripremljen kao prostor za buduće proširenje (npr. perzistencija rezervacija). Damage-service se prema S3 API-ju odnosi identično kao prema pravom AWS-u – razlikuje se samo `endpoint_url`.

2.8 Cross-Origin Resource Sharing

Uvođenjem gatewaya frontend i svi REST servisi poslužuju se s *istog ishodišta* (`:80`), pa zahtjevi iz preglednika više nisu *cross-origin* i CORS u praksi nije potreban. Servisi i dalje zadržavaju `CORSMiddleware` s permisivnom konfiguracijom (`allow_origins=["*"]`) kao sigurnosnu mrežu za izravan pristup tijekom razvoja (npr. Vite dev poslužitelj na `:5173` ili izravno gađanje servisa). Vrijedi prisjetiti se izvornog problema: dok su se frontend i servisi posluživali s različitih portova, svaki je zahtjev bio cross-origin prema SOP-u (Same-Origin Policy) i bez `CORSMiddleware`-a rezultirao bi greškom *CORS preflight failed*. U produkciji bi se `allow_origins` ograničio na konkretnu domenu frontenda.

3 Tehnologije

Svi backend servisi napisani su u **Python 3.10**. Za tri servisa koji izlažu REST API odabran je **FastAPI** – asinkroni web framework koji uz minimalan kod pruža automatsku validaciju podataka putem Pydantic modela, automatsko generiranje OpenAPI (Swagger) dokumentacije i visoke performanse temeljene na Starlette/Uvicorn stacku. GPS-tracker koristi **gRPC** s **Protocol Buffers** definicijama sučelja, što omogućuje strogo tipiziranu, binarno serijaliziranu komunikaciju s nižim overheadom od JSON-a.

Za asinkronu komunikaciju između servisa koristi se **Redis**, in-memory pohrana podataka koja ovdje funkcionira kao message broker. Pristup AWS resursima ostvaruje se putem **boto3** SDK-a, dok **LocalStack** omogućuje lokalno pokretanje S3 i DynamoDB servisa bez cloud infrastrukture. Autentifikacija se temelji na **JWT** tokenima implementiranim pomoću **PyJWT** biblioteke.

Frontend koristi **Vue.js 3** s Composition API-jem, **Vue Router** za upravljanje rutama i **Axios** kao HTTP klijent. Build proces odvija se putem **Vite** alata, a produkcijsko posluživanje preuzima **Nginx**. Svi servisi pakirani su u **Docker** kontejnere i orkestrirani

putem **Docker Compose-a**. Za centralizirano praćenje grešaka integriran je **Sentry** sa `StarletteIntegration` na backendu i `browserTracingIntegration` na frontendu.

4 Funkcionalnosti

4.1 Autentifikacija i autorizacija

Tok autentifikacije započinje unosom korisničkog imena i lozinke na Login stranici. Frontend šalje `POST /login` zahtjev Auth-servisu koji provjerava kredencijale i, ako su ispravni, vraća JWT token s `sub` claimom (korisničko ime) i `exp` claimom (istek za 1 sat). Frontend pohranjuje token u `localStorage` i od tog trenutka ga automatski uključuje u svaki zahtjev prema zaštićenim servisima putem Axios interceptora. Vue Router dodatno štiti zaštićene rute – `navigation guard` provjerava prisutnost tokena prije svakog prijelaza na zaštićenu stranicu.

4.2 Rezervacija vozila

Korisnik na stranici za rezervacije odabire jedno od pet dostupnih vozila (BMW X5, Audi A6, Mercedes C200, VW Golf, Škoda Octavia), unosi e-mail adresu, datume preuzimanja i vraćanja te opcionalne podatke poput broja putnika i napomena. Backend API trenutno prihvća samo dva query parametra – `car_id` i `user_email` – dok dodatna polja (datumi, broj putnika, telefon, napomena) ostaju na razini prezentacijskog sloja i prikazuju se u potvrdi; ovakva jasna razdvojenost ostavlja prostor za proširenje API-ja bez promjene UI-a.

Booking-service prima zahtjev, generira UUID identifikator, konstruira JSON poruku i stavlja je u Redis red. Korisniku se odmah prikazuje potvrda s identifikatorom i statusom `confirmed`. U pozadini, Mail-worker preuzima poruku iz reda i simulira slanje potvrdnog e-maila. Ovakva asinkrona obrada osigurava da korisnik nikada ne čeka na sporedne operacije poput slanja e-maila.

4.3 Evidencija šteta

Prijava štete svodi se na odabir fotografije oštećenja u jednom od podržanih formata (JPG, PNG, GIF) i upload putem web sučelja. Frontend šalje datoteku kao `multipart/form-data` na Damage-service koji je prosljeđuje u S3 bucket. Korisnik dobiva potvrdu sa statusom `pending`, signalizirajući da je slika zaprimljena i čeka obradu. U produkcijskom okruženju, S3 event bi pokrenuo Lambda funkciju koja bi mogla provesti automatsku analizu slike.

4.4 GPS praćenje

GPS-tracker prima lokacijske podatke putem gRPC poziva. Svaki poziv sadrži identifikator vozila i GPS koordinate. Servis bilježi primljene lokacije, čime pruža temelj za praćenje flote vozila u stvarnom vremenu. Korištenje gRPC-a umjesto REST-a opravdano je prirodom podataka – GPS koordinate su male, strukturirane poruke koje se šalju u visokoj frekvenciji, a binarna serijalizacija Protocol Buffersa i multipleksiranje HTTP/2 protokola značajno smanjuju mrežni overhead.

4.5 Praćenje grešaka

Svaki servis integriran je sa Sentry platformom za praćenje grešaka. Sentry konfiguracija čita se iz varijabli okruženja: `SENTRY_DSN` (ciljni projekt), `SENTRY_ENV` (okruženje, default `development`) te `SENTRY_TRACES_SAMPLE_RATE` (udio zahtjeva koji se uzorkuje za performance tracing, default `0.0`). Inicijalizacija je omotana u `try/except` blok pa se servisi čisto pokreću i u odsustvu DSN-a – Sentry tada tiho postaje no-op.

Backend servisi koriste `StarletteIntegration` (za REST servise), `StdlibIntegration` i `LoggingIntegration`, dok frontend koristi `browserTracingIntegration` za praćenje performansi i `replayIntegration` za snimanje korisničkih sesija prilikom grešaka. Zastavica `send_default_pii=True` omogućuje da Sentry automatski prikuplja osnovne podatke o korisniku (IP, user-agent, email ako je postavljen).

Svaki servis nudi `GET /__debug/crash` endpoint za namjerno izazivanje iznimke i testiranje Sentry pipeline-a. Mail-worker, koji nema HTTP sučelje, podržava udaljeni crash signal putem Redis ključa `crash_mail_worker`; Booking-service postavlja taj ključ preko vlastitog `POST /__debug/crash-mail-worker` endpointa, a worker ga obrađuje na idućoj BLPOP iteraciji. Frontend komponenta `CrashControls` objedinjuje sve ove akcije u jednostavno sučelje s četiri gumba.

5 Komunikacijski protokoli

Odabir komunikacijskog protokola jedno je od prvih pitanja pri dizajnu distribuiranog sustava i nema jednog pravog odgovora. U RS-Rentlyju koristim tri protokola jer svaki bolje odgovara drugačijem scenariju.

5.1 REST API (HTTP/JSON)

Auth-service, Booking-service i Damage-service komuniciraju s frontendom putem REST API-ja. Zahtjevi i odgovori razmjenjuju se u JSON formatu, a FastAPI automatski generira interaktivnu Swagger dokumentaciju na `/docs` putanji svakog servisa. REST je odabran za ove servise jer je intuitivno razumljiv, izvrstan za CRUD operacije i nativno podržan u web preglednicima. Tablica 1 prikazuje pregled svih REST endpointa.

Tablica 1: Pregled REST API endpointa

Servis	Metoda	Endpoint	Opis
Auth	POST	/login	Prijava korisnika, vraća JWT token
Auth	GET	/verify	Provjera valjanosti JWT tokena
Auth	GET	/__debug/crash	Simulirani pad servisa (Sentry test)
Booking	POST	/bookings	Kreiranje nove rezervacije vozila
Booking	GET	/__debug/crash	Simulirani pad servisa (Sentry test)
Booking	POST	/__debug/crash-mail-worker	Postavlja Redis signal za pad Mail-workera
Damage	POST	/upload-damage	Upload slike oštećenja na S3
Damage	GET	/__debug/crash	Simulirani pad servisa (Sentry test)

5.2 gRPC (Protocol Buffers)

GPS-tracker koristi gRPC – framework za udaljene pozive procedura temeljen na HTTP/2 transportu i Protocol Buffers serijalizaciji. Sučelje servisa definirano je .proto datotekom iz koje se automatski generira klijentski i serverski kod:

```

syntax = "proto3";

service GPSTracker {
  rpc SendLocation (LocationUpdate) returns (Empty) {}
}

message LocationUpdate {
  string car_id = 1;
  double latitude = 2;
  double longitude = 3;
}

message Empty {}

```

Listing 1: Definicija gRPC sučelja (gps.proto)

U usporedbi s REST-om, binarna serijalizacija daje manje poruke od JSON-a, HTTP/2 omogućuje multipleksiranje poziva, a .proto shema sprječava greške pri serijalizaciji koje bi se inače teže pronašle.

5.3 Redis Message Queue

Komunikacija između Booking-servisa i Mail-workera odvija se putem Redis liste `email_queue`. Booking-service dodaje poruke naredbom `RPush`, a Mail-worker ih čita blokirajućom naredbom `BLPOP` s timeoutom od 5 sekundi. Ovo je klasičan producer-consumer obrazac koji donosi tri ključne prednosti: vremensko odvajanje (producent i konzument ne moraju raditi istovremeno), apsorpcija opterećenja (Redis služi kao buffer pri naglim skokovima) i otpornost na padove (poruke preživljavaju restart workera).

6 API Gateway i usmjeravanje prometa

U izvornoj inačici frontend je komunicirao izravno s pojedinim servisima na zasebnim portovima (:8000, :8001, :8002). Takva izravna sprega ima dva problema: klijent mora poznavati topologiju i portove svakog servisa, a svaki je poziv *cross-origin* (vidi potpoglavlje o CORS-u). Stoga je uveden **Traefik** (verzija 3.1) kao *reverse proxy* i jedinstvena ulazna točka cijelog sustava.

Sav promet sada ulazi kroz gateway na portu :80, koji ga usmjerava prema odgovarajućem servisu na temelju putanje (**PathPrefix**):

Tablica 2: Pravila usmjeravanja na gatewayu

Ulazna putanja	Servis	Napomena
/api/auth/*	auth-service	StripPrefix uklanja /api/auth
/api/booking/*	booking-service	StripPrefix uklanja /api/booking
/api/damage/*	damage-service	StripPrefix uklanja /api/damage
/ (ostalo)	frontend	SPA, najniži prioritet (hvata sve ostalo)
gRPC :50051	gps-tracker	zasebna ulazna točka, shema h2c

Traefik otkriva servise dinamički, čitanjem Docker labela (`provider.docker`, uz `exposedByDefault=false` – izlažu se samo servisi s eksplicitnim `traefik.enable=true`). Time nema potrebe za ručnom statičkom konfiguracijom rutiranja: dodavanjem nove replike ili servisa gateway ga automatski uključi. Traefik dashboard dostupan je na portu :8080 i nudi pregled aktivnih ruta, servisa i replika u stvarnom vremenu, a izložene su i Prometheus metrike.

Ključna posljedica je da se frontend i svi REST servisi sada poslužuju s *istog ishodišta* (gateway na :80), pa CORS više nije potreban – `api.js` koristi relativne putanje (`/api/auth`, `/api/booking`, `/api/damage`) umjesto apsolutnih URL-ova s portovima.

7 Horizontalno skaliranje

Svi aplikacijski servisi projektirani su kao *stateless* – ne čuvaju lokalno stanje, nego se oslanjaju na dijeljene resurse (Redis, S3) ili na samodostatne tokene (JWT). Zahvaljujući tome svaki se servis može pokrenuti u proizvoljnom broju identičnih replika, a gateway promet ravnomjerno raspoređuje među njima (*round-robin*).

Preduvjet za skaliranje bilo je uklanjanje fiksni *host* portova s aplikacijskih servisa: dok je svaki servis objavljivao konkretan port na domaćinu, pokretanje druge replike rezultiralo bi sukobom (*address already in use*). Nakon prelaska na otkrivanje preko gatewaya, servisi izlažu portove samo unutar Docker mreže, pa replikacija postaje trivijalna:

```
docker compose up --build -d \
  --scale auth-service=3 \
  --scale booking-service=3 \
  --scale damage-service=3 \
  --scale mail-worker=3 \
  --scale gps-tracker=3
```

Listing 2: Skaliranje na proizvoljan broj replika

Tablica 3: Mehanizam skaliranja po servisu

Servis	Kako se raspoređuje promet
auth / booking / damage	Traefik round-robin po HTTP-u (REST)
mail-worker	<i>competing consumers</i> – svaka poruka iz reda (BRPOPLPUSH) pripadne točno jednom workeru
gps-tracker	Traefik gRPC (h2c) load balancing
redis / localstack	singletoni s dijeljenim stanjem – namjerno se ne skaliraju

`docker-compose.yaml` već u zadanim postavkama diže po dvije replike svakog servisa (`deploy.replicas: 2`), čime je horizontalna skalabilnost vidljiva odmah pri pokretanju.

8 Otpornost na pogreške (resilience)

Skaliranje bez mehanizama oporavka od grešaka bilo bi nepotpuno – u raspodijeljenom sustavu kvarovi pojedinih čvorova i mreže su *očekivani*, ne iznimni. Sustav stoga primjenjuje obrasce otpornosti na dvije razine: na gatewayu i unutar samih servisa.

8.1 Razina gatewaya

- **Health-check load balancera** – Traefik svakih 10 sekundi provjerava `/health` endpoint svake replike; replika koja ne odgovara automatski izlazi iz rotacije i vraća se tek kad ponovno postane zdrava.
- **Retry** – na mrežnu grešku prema replici gateway automatski preusmjerava zahtjev na sljedeću zdravu repliku istog servisa (*failover*).
- **Circuit breaker** – kada udio mrežnih grešaka prema servisu prijeđe 30% ($NetworkErrorRatio > 0,30$), sklopka se otvara i gateway odmah vraća 503 (*fail-fast*) umjesto da zatrpava servis koji ionako pada. Sklopka se sama zatvori kad se servis oporavi.
- **Timeouts** – ograničenja na uspostavu i trajanje konekcije prema backendima sprječavaju da zaglavljeni servis trajno drži otvorene resurse.

8.2 Razina servisa

- **booking** → **Redis**: konekcija ima socket timeout; upis u red štiti se ponavljanjem uz *eksponencijalni backoff* (0,1 → 0,2 → 0,4 s) te *in-process* circuit breakerom. Kod nedostupnog Redisa servis vraća 503 s **Retry-After** zaglavljem (a ne 500), čime se ostvaruje *graceful degradation*.
- **damage** → **S3**: boto3 klijent konfiguriran je s connect/read timeoutima i automatskim ponavljanjem (*standard* mode, do tri pokušaja uz backoff).
- **mail-worker** (pouzdana razmjena poruka):
 - *Reliable queue* – poruka se atomarno premješta iz glavnog reda u **processing** listu naredbom BRPOPLPUSH. Ako worker padne usred obrade, poruka preživi u **processing** listi i pri sljedećem se pokretanju vraća u red (*crash recovery*) – nema izgubljene obrade.

- *Dead-letter queue* (`email_queue_dead`) – poruke koje se ne mogu obraditi (neispravan JSON, nedostaje e-mail) ili su iscrpile dopušteni broj pokušaja odlažu se u DLQ umjesto da blokiraju red.
- *Retry brojač* – prolazne greške slanja ponavljaju se do `MAX_ATTEMPTS` (3), nakon čega poruka ide u DLQ.
- *Idempotentnost* – skup obrađenih identifikatora (`processed_bookings`) osigurava da se mail za istu rezervaciju ne pošalje dvaput, čak i ako se poruka ponovno pojavi nakon oporavka (semantika *at-least-once* + idempotentni potrošač).

Ovi mehanizmi zajedno čine da pojedinačni kvar (pad replike, kratkotrajna nedostupnost Redisa, neispravna poruka) ne ruši cijeli sustav, nego se lokalizira i automatski sanira.

9 Struktura projekta

```
rs-rently/
|
|- auth-service/
|  |- main.py # JWT autentifikacija (FastAPI)
|  |- Dockerfile
|
|- booking-service/
|  |- main.py # Rezervacije + Redis queue (FastAPI)
|  |- Dockerfile
|
|- damage-lambda/
|  |- main.py # S3 upload (FastAPI)
|  |- lambda-function.py # AWS Lambda handler
|  |- Dockerfile
|
|- mail-worker/
|  |- main.py # Redis consumer, simulacija e-maila
|  |- Dockerfile
|
|- gps-tracker/
|  |- main.py # gRPC server
|  |- gps.proto # Protocol Buffers definicija
|  |- Dockerfile
|
|- frontend/
|  |- src/
|  |  |- components/
|  |  |  |- Login.vue # Prijava korisnika
|  |  |  |- Dashboard.vue # Pregled sustava
|  |  |  |- Bookings.vue # Kreiranje rezervacija
|  |  |  |- DamageUpload.vue # Upload slika steta
|  |  |  |- CrashControls.vue # Testiranje padova
|  |  |- api.js # Axios instance + interceptori
|  |  |- main.js # Vue app, router, Sentry init
|  |  |- App.vue # Root komponenta s navigacijom
|  |- nginx.conf # SPA routing, gzip, cache
|  |- vite.config.js
|  |- Dockerfile # Multi-stage build (Node + Nginx)
|  |- package.json
|
```

```
| - docker-compose.yaml # Orkestracija svih servisa
| - start.sh
| - README.md
```

Listing 3: Direktorijska struktura projekta

Svaki backend servis sadrži vlastiti `main.py` i `Dockerfile`. Zahvaljujući toj organizaciji, servise je moguće razvijati i deployati neovisno. Frontend koristi multi-stage Docker build – prva faza kompajlira Vue aplikaciju, a druga kopira generirane datoteke u Nginx kontejner.

10 Docker i pokretanje

Cijeli sustav orkestriran je jednom `docker-compose.yaml` datotekom koja definira osam servisa (šest aplikacijskih i dva infrastrukturna) povezanih zajedničkom Docker bridge mrežom `rent_network`. Unutar te mreže, servisi se referenciraju po imenu – primjerice, Booking-service pristupa Redisu na hostu `redis`, a Damage-service pristupa LocalStacku na `localstack:4566`. Varijable okruženja poput `SENTRY_DSN` i `SENTRY_ENV` prosljeđuju se iz host sustava putem `${...}` sintakse.

Za izgradnju i pokretanje cijelog sustava dovoljne su dvije naredbe:

```
docker-compose up --build # prva izgradnja
docker-compose up # svako sljedeće pokretanje
```

Konfiguracija Sentryja. Backend servisi očekuju varijable `SENTRY_DSN`, `SENTRY_ENV` i `SENTRY_TRACES_SAMPLE_RATE` u root `.env` datoteci. Frontend (Vite) čita `VITE_SENTRY_DSN` iz `frontend/.env` tijekom build faze – zbog toga je nakon promjene DSN-a potrebno pokrenuti `docker-compose up -build` kako bi se frontend ponovno kompajlirao s novom vrijednošću.

Nakon pokretanja, sustav je dostupan na sljedećim adresama:

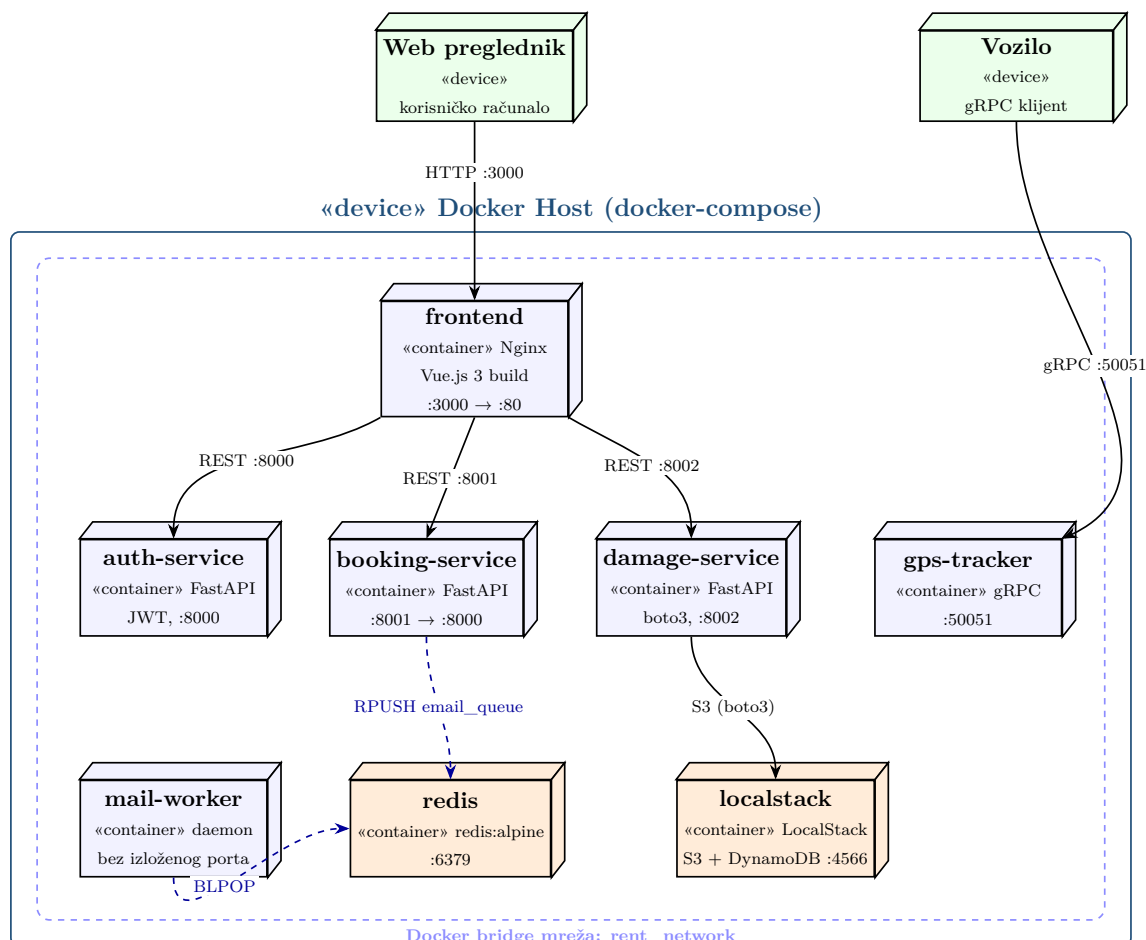
Tablica 4: Mrežna konfiguracija servisa

Servis	Port	Protokol	Napomena
Frontend	3000	HTTP	Nginx, SPA routing
Auth-service	8000	HTTP	Swagger na <code>/docs</code>
Booking-service	8001	HTTP	Swagger na <code>/docs</code>
Damage-service	8002	HTTP	Swagger na <code>/docs</code>
GPS-tracker	50051	gRPC	Protocol Buffers
LocalStack	4566	HTTP	DynamoDB + S3
Redis	6379	TCP	Message broker

Za razvoj frontenda s automatskim osvježavanjem koristi se Vite dev server (`npm run dev` u `frontend/` direktoriju), koji aplikaciju servira na portu 5173.

11 Dijagram raspodjele – razvojno okruženje

Dijagram na slici 1 prikazuje fizičku raspodjelu (deployment) sustava u razvojnom okruženju. Svaki servis izvodi se kao zaseban Docker kontejner – u UML notaciji raspodjele prikazan kao izvršni čvor («container») s naznakom slike i izloženog porta. Svi kontejneri smješteni su unutar Docker bridge mreže `rent_network`, koja je pak ugniježdjena u jedan fizički host («device» Docker Host). Vanjski klijenti – web preglednik korisnika i vozilo kao gRPC klijent – nalaze se izvan hosta i komuniciraju kroz objavljene portove. Bridovi su označeni konkretnim komunikacijskim protokolom i portom.



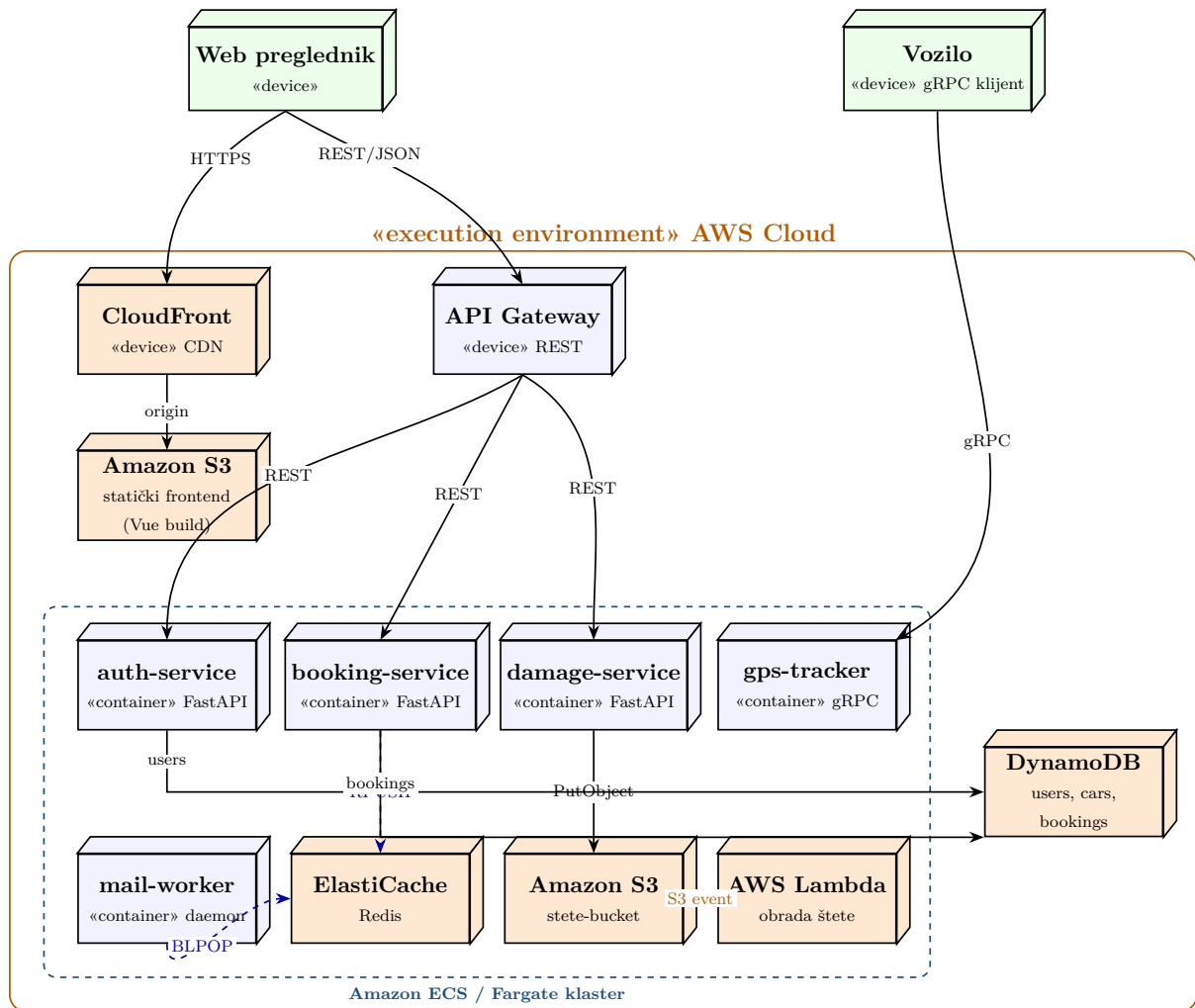
Slika 1: Dijagram raspodjele – razvojno okruženje (Docker Compose + LocalStack)

Cijeli sustav izvodi se na jednom fizičkom čvoru, a izolacija servisa ostvaruje se kontejnerizacijom umjesto zasebnim strojevima. To je tipično za razvoj: jedna `docker-compose up` naredba podiže cjelokupnu topologiju. Tri REST veze (:8000, :8001, :8002) idu od frontenda prema aplikacijskim servisima, dvije isprekidane veze prikazuju asinkroni Redis red (RPUSH/BLPOP) između Booking-servisa i Mail-workera, a Damage-service piše fotografije u S3 koji simulira LocalStack. LocalStack na ovom čvoru zamjenjuje stvarne AWS servise – razlika prema produkciji vidljiva je na sljedećem dijagramu.

12 Dijagram raspodjele – produkcijsko okruženje

Dijagram na slici 2 prikazuje ciljnu (produkcijsku) raspodjelu sustava na AWS infrastrukturi. Ključna razlika u odnosu na razvojno okruženje jest da svaki infrastrukturni element koji

LocalStack lokalno simulira ovdje postaje stvarni upravljani AWS servis: **localstack** se razlaže na **Amazon S3** (pohrana fotografija), **AWS Lambda** (obrada okinuta S3 eventom) i **Amazon DynamoDB** (perzistencija), Redis postaje **Amazon ElastiCache**, a aplikacijski kontejneri izvode se na **Amazon ECS / Fargate** klasteru iza **Amazon API Gateway-a**. Frontend se servira kao statički build iz S3 bucketa putem **CloudFront** CDN-a. Topologija veza ostaje identična – mijenja se samo izvršno okruženje čvorova, što je izravna posljedica mikroservisne arhitekture.



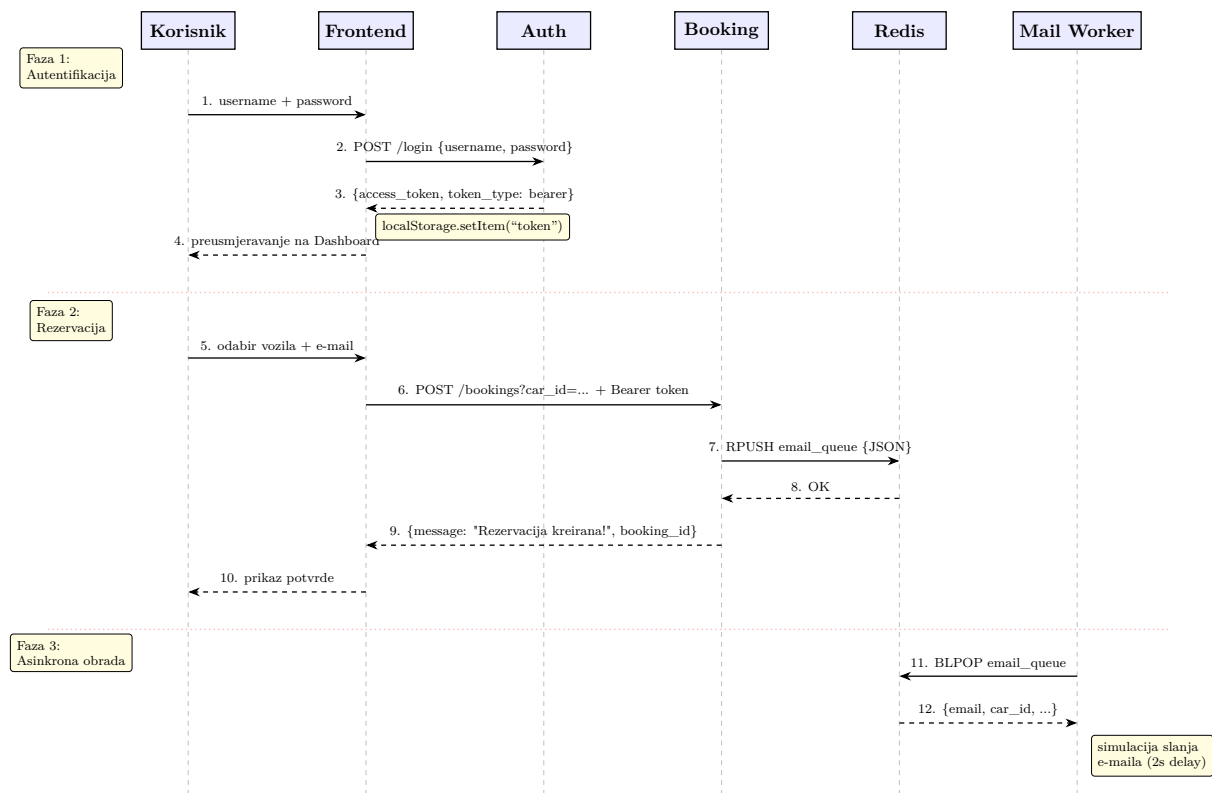
Slika 2: Dijagram raspodjele – ciljno produkcijsko okruženje (AWS)

Usporedba dvaju dijagrama raspodjele izravno pokazuje vrijednost LocalStacka: razvojni i produkcijski dijagram dijele istu logičku topologiju, pa migracija na AWS ne zahtijeva preradu aplikacijskog koda nego samo zamjenu `endpoint_url`-a i izvršnih čvorova. Točkasta veza «S3 event» prikazuje obrazac koji u razvoju ostaje neaktivan (Lambda se ne okida), a u produkciji omogućuje automatsku obradu uploadanih fotografija šteta.

13 Sekvencijski dijagram

Slika 3 prikazuje dva osnovna toka sustava. Prvi tok (koraci 1–4) ilustrira autentifikaciju: korisnik unosi kredencijale, Frontend ih prosljeđuje Auth-servisu koji vraća JWT token,

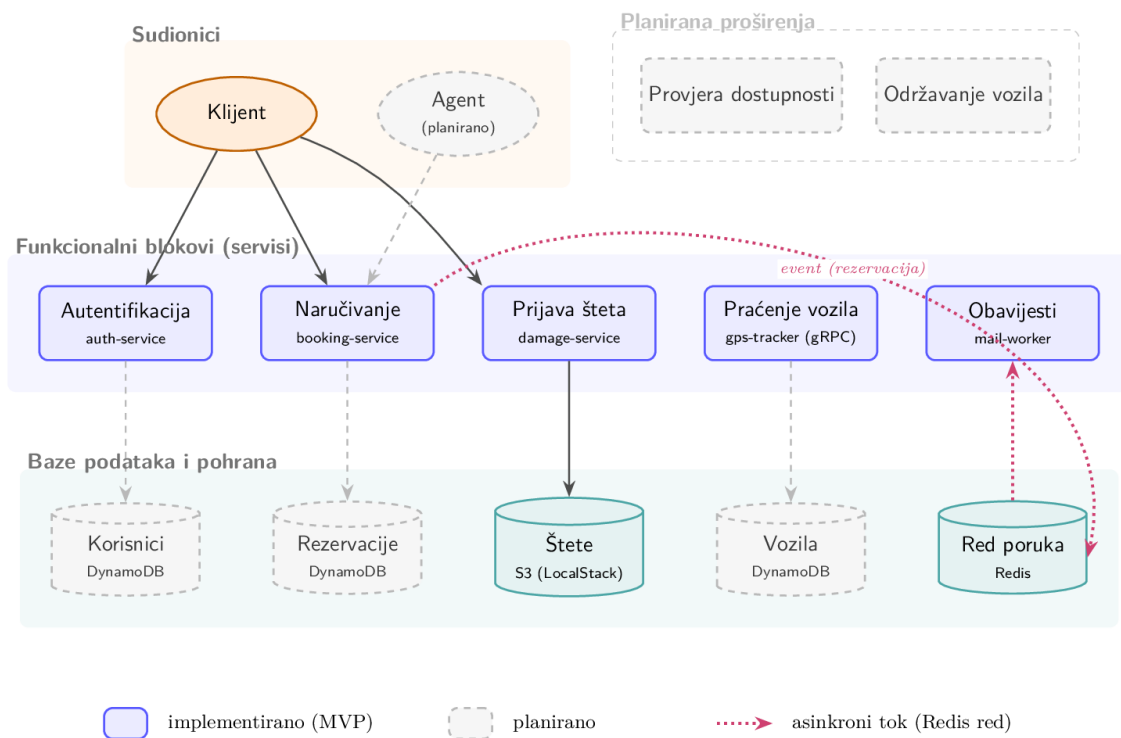
a korisnik biva preusmjeren na Dashboard. Drugi tok (koraci 5–11) ilustrira kreiranje rezervacije s asinkronom obradom: Booking-service prima zahtjev, gura poruku u Redis red i odmah vraća potvrdu, dok Mail-worker neovisno preuzima poruku i šalje e-mail.



Slika 3: Sekvencijski dijagram – autentifikacija, rezervacija i asinkrona obrada

14 Konceptualni model domene

U ranoj fazi planiranja sustava izrađen je konceptualni dijagram koji prikazuje cjelovitu domenu rent-a-car poslovanja – sve sudionike, funkcionalne blokove i baze podataka koje bi dovršeni sustav trebao obuhvatiti. Taj model je šire postavljen od trenutnog MVP-a i služi kao *roadmap* za buduće iteracije; važno je jasno razlikovati što je implementirano u ovoj verziji, a što je zadržano kao prostor za daljnji razvoj.



Slika 4: Konceptualni dijagram domene – ciljno stanje sustava

Tablica 5 izravno mapira svaki element dijagrama na trenutni kod, označavajući što je dovršeno (✓), a što je predviđeno za iduću iteraciju (×).

Tablica 5: Pokrivenost konceptualnog modela u MVP implementaciji

Komponenta iz plana	MVP	Napomena
Autentifikacija korisnika	✓	<code>auth-service</code> , JWT
Realtime praćenje vozila	✓	<code>gps-tracker</code> preko gRPC-a
Naručivanje vozila (klijent)	✓	<code>booking-service</code>
Frontend sučelje za klijente	✓	Vue SPA (<code>Bookings</code> , <code>DamageUpload</code>)
Sustav za prijavu šteta	✓	<code>damage-service</code> + S3
Frontend za prijavu šteta	✓	komponenta <code>DamageUpload.vue</code>
Sustav za slanje maila	✓	<code>mail-worker</code> + Redis queue
Baza korisnika	×	Kredencijali trenutno hardkodirani (<code>admin/admin</code>)
Baza vozila	×	Vozila su statička lista u <code>Bookings.vue</code>
Baza rezervacija	×	Rezervacije prolaze kroz Redis queue bez perzistencije; DynamoDB u LocalStacku pripremljen za ovu namjenu
Održavanje vozila	×	Domenski koncept, nije realiziran kao servis
Provjera dostupnosti vozila	×	<code>booking-service</code> trenutno ne provjerava zauzetost termina
Frontend sučelje agenta	×	Postoji samo jedinstveni frontend; agentski pogled predviđen uz uvođenje baze rezervacija
Poveznica šteta ↔ rezervacija	×	<code>damage-service</code> uploada fotografiju bez referenciranja konkretne rezervacije

14.1 Opseg MVP-a

Fokus ove verzije je demonstracija *komunikacijskih obrazaca* distribuiranog sustava – REST, gRPC i asinkrona obrada putem message queue-a – a ne potpuna poslovna logika rent-a-car operacije. Zbog toga su komponente koje zahtijevaju perzistentnu pohranu svjesno pojednostavljene: umjesto baza podataka korištene su hardkodirane vrijednosti i in-memory strukture, čime se zadržava jasnoća protoka poruka između servisa i izbjegava komplikacija vezana uz sheme, migracije i transakcije.

14.2 Predviđena proširenja

Prirodni sljedeći korak je uvođenje perzistencije. DynamoDB je već konfiguriran u LocalStacku, pa bi dodavanje tablica `users`, `cars` i `bookings` zahtijevalo samo ukidanje hardkodiranih vrijednosti u `auth-service`-u i `booking-service`-u te dodavanje čitanja i pisanja preko `boto3` klijenta. Nakon što `booking-service` dobije pristup bazi rezervacija, prirodno slijede: provjera dostupnosti termina, povezivanje uploadanih fotografija šteta s konkretnim rezervacijama te izrada zasebnog *agentskog* sučelja koje bi dohvaćalo i ažuriralo te zapise. Održavanje vozila može se realizirati kao novi mikroservis s vlastitim schedulerom – bez ikakvih izmjena u postojećim servisima, jedino dodavanjem novog čvora u Docker mrežu.

Ova postupnost ujedno demonstrira prednost mikroservisne arhitekture – svaki od tih koraka može se implementirati neovisno, bez dodirivanja ostatka sustava.

15 Zaključak

Kroz ovaj projekt pokazao sam kako se jedan veći sustav može razbiti na neovisne servise koji međusobno komuniciraju putem mreže. Svaki servis ima jednu odgovornost i može se razvijati i deployati neovisno.

Korištenjem tri različita protokola – REST, gRPC i Redis queue – vidljivo je da nema jednog rješenja koje odgovara svim scenarijima. REST je jednostavan za CRUD i standardnu komunikaciju, gRPC ima smisla gdje je overhead JSON-a problematičan, a message queue rješava asinkronu obradu bez blokiranja.

Sustav se pokreće jednom naredbom i ne zahtijeva cloud infrastrukturu zahvaljujući LocalStacku. Sentry integracija pokrila je sve razine, od backenda do frontenda, što bi u produkcijskom okruženju bila osnova za dijagnostiku. Arhitektura je relativno lako proširiva – novi servis dodaje se kao novi kontejner u mrežu, bez diranja postojećih.